



BANK MANAGEMENT SYSTEM

OBJECT-ORIENTED PROGRAMMING (OOP) PROJECT

PERPARED BY:

EL MOKHTAR FERTIT,

SAID LOUHAM,

MOHAMED YASSINE

ASNUS

SUPERVISED BY:

PROFESSEUR: EL OUAFAFI RAJAA

ACADEMIC YEAR:

2025/2026

1. Group Members and Roles:

This project was developed by three group members. Each member participated in the analysis, design, implementation, and testing of the **Bank Management System** application. The group members are listed in the report example cover page as **EL MOKHTAR FERTIT**, **MOHAMED YASSINE ASNUS**, and **SAID LOUHAM**.

Group Member	Role in the Project
EL MOKHTAR FERTIT	Worked on the main project structure, the controller part, and the connection between the application logic and the GUI. He also helped with GitHub organization and report preparation.
MOHAMED YASSINE ASNUS	Worked on the graphical user interface using Java Swing, including windows, panels, buttons, forms, and user interaction.
SAID LOUHAM	Worked on the model classes and testing part, such as customers, accounts, transactions, exceptions, and unit tests.

2. Project Description

The **Bank Management System** is a Java desktop application created to simulate the main operations of a small bank. The application is built with **Java Swing**, and it allows an employee to log in and manage bank customers, accounts, and transactions from a graphical interface. The system includes features such as customer creation, customer search, checking and savings accounts, deposits, withdrawals, transfers, transaction history, and dashboard totals.

The main objective of this application is to replace manual bank management with a simple computerized system. Instead of keeping customer and account information on paper, the application stores data digitally and allows the user to perform banking operations in an organized way. For example, an employee can create a customer, open an account, deposit money, withdraw money, transfer money between accounts, and check previous transactions. The system also uses a database with SQLite to keep account data saved between application sessions.

This project was also developed to apply the main concepts learned in Java Object-Oriented Programming. It uses different packages for models, services, controller, repository, GUI, exceptions, and threads. The GUI sends user actions to the BankController, then the controller communicates with the service classes to apply business rules. The project also demonstrates important course topics such as custom exceptions, collections, threads, JDBC persistence, Swing GUI, logging, and JUnit tests.

3. Application Objectives

The main objective of the Bank Management System is to create a simple Java desktop application that helps manage basic banking operations in an organized way. The application was developed to simulate how a small bank system can manage customers, accounts, and transactions using Object-Oriented Programming principles.

Objective	Explanation
Manage customers	The application allows the employee to add, view, search, and manage customer information.
Manage bank accounts	The system supports account creation and account details, such as account number, balance, and account type.
Perform transactions	The application allows basic banking operations such as deposit, withdrawal, and transfer between accounts.
Keep transaction history	Every important operation is saved as a transaction so the employee can follow the activity of an account.
Use a graphical interface	The project uses Java Swing to make the application easier to use instead of using only the console.
Apply OOP concepts	The project applies classes, objects, inheritance, encapsulation, exceptions, collections, and separation between packages.
Improve data organization	The application separates the code into model, service, controller, GUI, repository, exception, and thread packages.
Practice real project structure	The project helps us understand how a Java application can be structured in a clean and maintainable way.

4. Main Functionalities of the Application

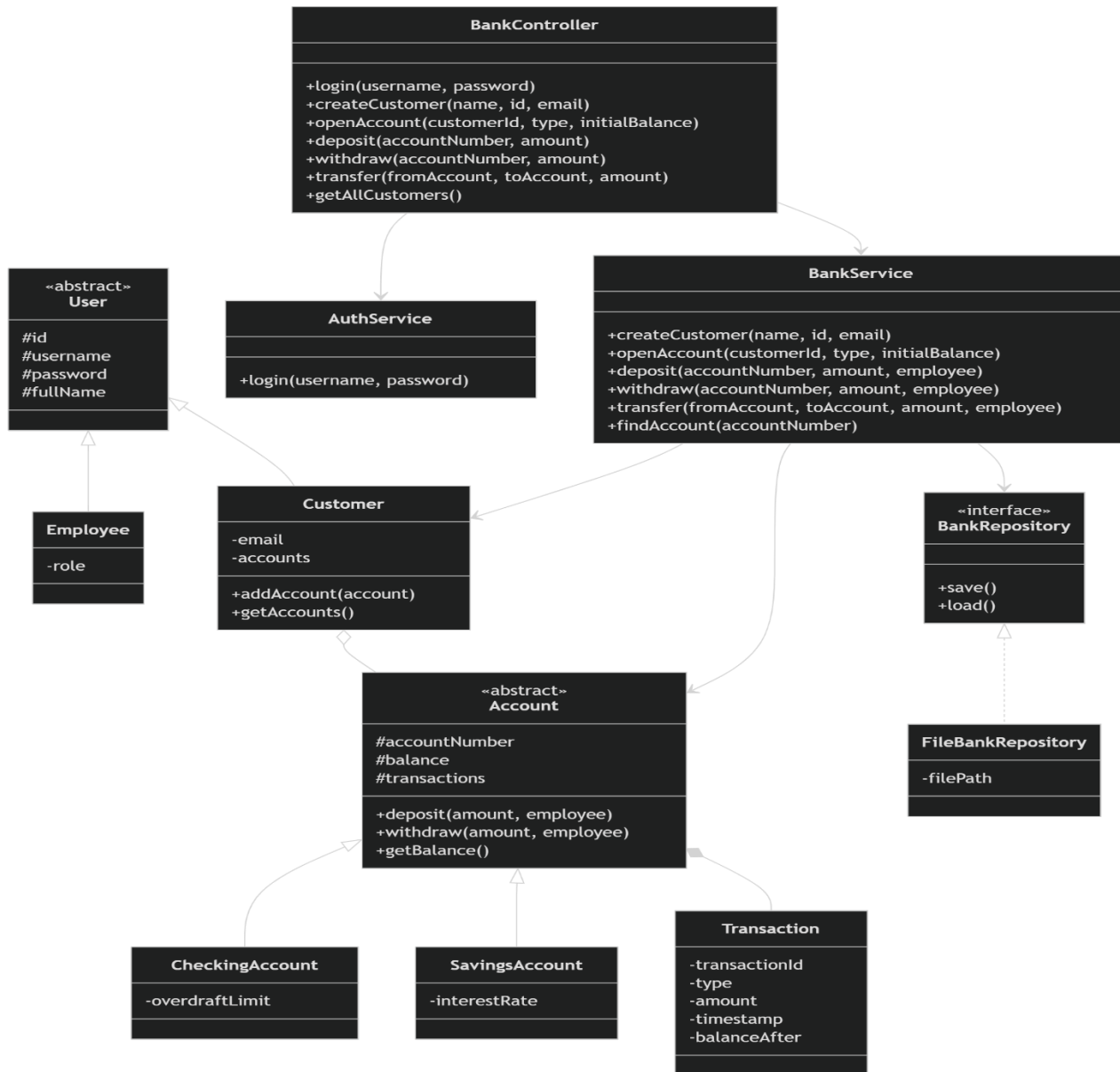
The **Bank Management System** allows an employee to manage the main operations of a bank from a graphical interface. After logging in, the employee can access the main window and use different panels to manage customers, accounts, and transactions. The system provides customer management features such as adding customers, viewing their details, and selecting their accounts.

The application also supports banking operations such as **deposit**, **withdrawal**, and **transfer**. Each operation updates the account balance and creates a transaction record. This makes it possible to keep a history of all important operations performed in the system. The application also uses validation and custom exceptions to prevent incorrect operations, such as withdrawing more money than the available balance.

Another important functionality is the organization of the application into different parts. The GUI handles user interaction, the controller connects the interface with the logic, the service classes apply business rules, and the repository is responsible for data saving. This structure makes the application clearer, easier to maintain, and closer to a real Java desktop project.

5. Class Diagram and Main Class Relationships:

The Bank Management System is organized around several main classes. The GUI classes communicate with the BankController, and the controller uses service classes to manage the application logic.



The Main class starts the program and opens the login window. After login, the user accesses the main interface through MainFrame. The interface does not directly manage the data; it sends requests to BankController.

The BankController calls BankService, which contains the main banking logic such as creating customers, creating accounts, deposit, withdrawal, and transfer. The model classes Customer, Account, and Transaction represent the main data of the system.

The project also uses BankException to manage errors, and thread classes such as TransactionWorker and AlertMonitor to process some tasks in the background.

6. Use of Course Topics in the Code

1. Exceptions:

In the **Bank Management System**, exceptions are used to handle errors in a clean way without stopping the application suddenly. The project uses a custom exception class called `BankException`. This exception is used when a banking operation is not valid, for example when the user enters an invalid amount, tries to withdraw more money than the account balance, or searches for an account that does not exist.

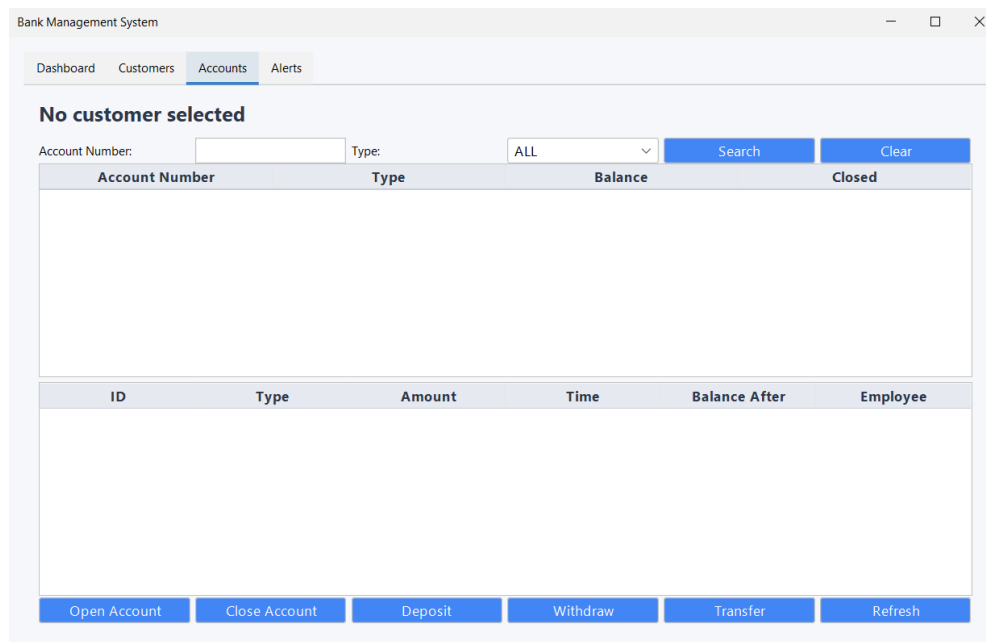
Using exceptions makes the code safer and easier to understand. Instead of writing error messages everywhere, the system throws a `BankException`, and the GUI catches it to show a clear message to the user.

```
BMS > src > main > java > com > bank > exception > J BankException.java
1  package com.bank.exception;
2
3  public abstract class BankException extends Exception {
4      public BankException(String message){
5          super(message);
6      }
7      public BankException(String message,Throwable cause){
8          super(message,cause);
9      }
10 }
```

2. Graphical User Interface GUI / Swing

The Bank Management System uses Java Swing to create a desktop graphical interface. Instead of using only the console, the user can interact with windows, buttons, text fields, tables, and panels. The GUI starts with `LoginFrame`, then after successful login the user accesses the main window `MainFrame`.

The GUI is separated into different panels, such as `DashboardPanel`, `CustomerListPanel`, `AccountDetailPanel`, `TransactionDialog`, and `AlertLogPanel`. This makes the interface more organized. The GUI does not do the banking logic directly; it sends user actions to the `BankController`, and the controller calls the service classes.



3. Collections:

In the **Bank Management System**, collections are used to store and manage several objects in memory, such as customers, accounts, and transactions. Instead of creating one variable for each customer or account, the application uses Java collections like List, ArrayList, and Map.

For example, customers can be stored in a list or a map, accounts can be linked to customers, and transactions can be saved in a transaction history. This makes it easier to add, search, update, and display data in the application.

4. Threads

In the **Bank Management System**, threads are used to run some tasks in the background without blocking the graphical interface. This is useful because the GUI must stay responsive while the system processes transactions or monitors alerts.

The project uses classes such as TransactionQueue, TransactionWorker, and AlertMonitor. The TransactionQueue stores transaction requests, and the TransactionWorker processes them in the background. The AlertMonitor can run separately to check important events and display alerts when needed.

5. Unit Tests:

In the **Bank Management System**, unit tests are used to check that important parts of the application work correctly. Tests help us verify the behavior of model classes and banking operations before using them in the GUI. For example, we can test account creation, deposit, withdrawal, and exception cases.

The project uses **JUnit** to write simple automated tests. Instead of testing everything manually from the interface, unit tests allow us to check if a method gives the expected result. This makes the project more reliable and helps detect errors early.

```
61  @Test
62  void createCustomer_shouldDelegateToBankService() {
63      Customer customer = controller.createCustomer(
64          id: "C001",
65          fullName: "Ahmed",
66          username: "ahmed",
67          password: "1234",
68          email: "ahmed@gmail.com"
69      );
70
71      assertNotNull(customer);
72      assertEquals(expected: "C001", customer.getId());
73      assertEquals(expected: "Ahmed", customer.getFullName());
74      assertEquals(expected: 1, controller.getAllCustomers().size());
75  }
76
```

7. Difficulties Encountered and Solutions:

One of the main difficulties was connecting the graphical interface with the application logic. At the beginning, it was not easy to decide where each operation should be placed. To solve this problem, we used the `BankController` class to connect the GUI with the service classes. This made the project more organized because the interface only sends requests, while the service classes handle the real banking logic.

Another difficulty was managing errors during banking operations. For example, the user could enter a negative amount, try to withdraw more than the available balance, or search for an account that does not exist. To solve this, we created a custom exception called `BankException`, which allows the application to show clear error messages without crashing.

We also faced difficulties with threads and background tasks. The solution was to use specific classes such as `TransactionQueue`, `TransactionWorker`, and `AlertMonitor`. These classes helped us process some tasks in the background while keeping the Swing interface responsive. Finally, we used unit tests to check important methods and reduce errors before using the application from the GUI.

8. Known Limitations and Missing Features

The **Bank Management System** works correctly for the main banking operations, but it still has some limitations. Some advanced features were not implemented because of time and project scope.

Limitation / Missing Feature	Explanation
Limited employee roles	The system has login, but it does not include many employee roles like admin, cashier, or manager.
Simple security	Authentication is basic and does not include password encryption or advanced access control.
Limited database features	The application saves data, but it does not include advanced database management or backup system.
No online banking	Customers cannot access their accounts online; only the employee uses the desktop application.
Simple reports	The system does not generate detailed PDF or Excel reports for transactions and accounts.
Basic GUI design	The interface is functional, but it can be improved with better styling and more modern design.
Limited testing	Unit tests were added for important parts, but not all GUI and service methods are fully tested.

9. Conclusion:

The **Bank Management System** is a Java desktop application developed to manage basic banking operations in a simple and organized way. The application allows employees to manage customers, accounts, and transactions such as deposit, withdrawal, and transfer.

This project helped us apply important Object-Oriented Programming concepts in a real application. We used classes and objects to represent customers, accounts, employees, and transactions. We also used Java Swing for the graphical interface, collections for storing data, exceptions for error handling, threads for background tasks, and unit tests to verify important methods.

In conclusion, the project gave us practical experience in building a structured Java application. Even if some advanced features are still missing, the system represents a good base that can be improved in the future with better security, more roles, advanced reports, and a more modern interface.